TRAINING AND SCALING LANGUAGE MODELS: FROM FIRST PRINCIPLES TO EFFICIENT SERVING

Module 1: Transformer from First Principles

Building a Mental Model of a Decoder-Only LM

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. Attention from first principles
2. The decoder block
3. Evidence before scale

Guiding question

What must happen between a token ID and a distribution over the next token?

What should be clear by the end?

1. Track the shape and meaning of every tensor in causal self-attention.
2. Explain why scaling and masking happen before the softmax.
3. Trace information through a pre-norm decoder block.
4. Name the tests that make a minimal implementation trustworthy.

First, the task: predict one token at a time

Start with the context: **The chef seasoned the**

Possible next tokens

soup	40%
sauce	27%
steak	17%
vegetables	12%
refrigerator	4%

Choose “soup” and predict again

New context: **The chef seasoned the soup**

Next tokens: **with** (55%), **and** (28%), or **.** (17%).

The probabilities are illustrative. In the demo, choose a continuation and watch the next prediction change.

Explore the interactive demo

PART 1

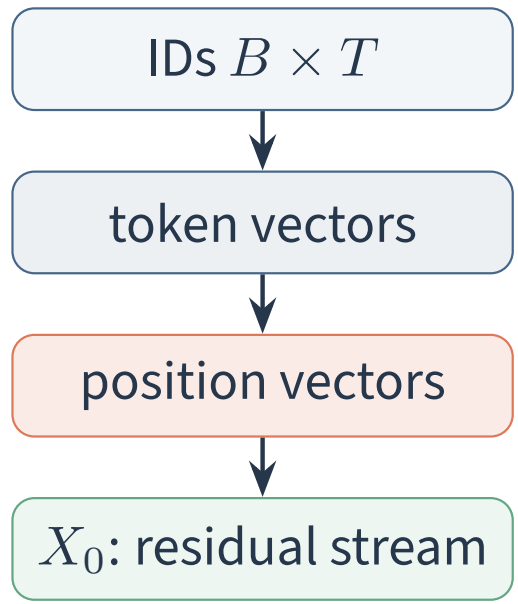
Attention from first principles

1. Attention from first principles
2. The decoder block
3. Evidence before scale

Tokens become points in a shared space

- Input IDs have shape $B \times T$.
- A lookup table $W_E \in \mathbb{R}^{V \times d}$ produces content vectors.
- Position information makes order visible to the model.

$$X_0 = W_E[\text{ids}] + P_{0:T} \quad \in \mathbb{R}^{B \times T \times d}$$



Attention is learned retrieval

Query
What does this position need?

$$Q = XW_Q$$

Key
What does each position offer?

$$K = XW_K$$

Value
What content should move?

$$V = XW_V$$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V$$

The distinction that matters

Queries and keys choose the mixture. Values supply the mixed information.

Source: Vaswani et al. and The Annotated Transformer

Why divide by $\sqrt{d_h}$?

Assume independent query and key components with unit variance.

$$q^\top k = \sum_{i=1}^{d_h} q_i k_i \qquad \mathrm{Var}(q^\top k) \approx d_h$$

Without scaling

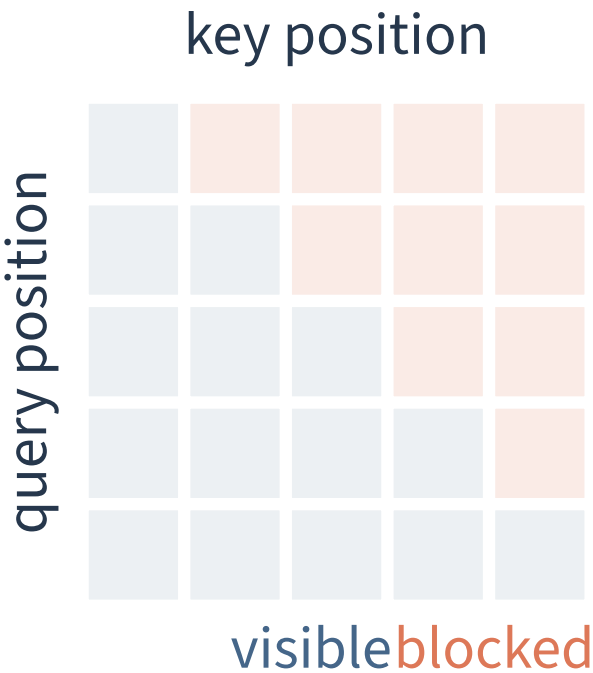
Wider heads create larger logits. Softmax saturates and gradients become poorly conditioned.

With scaling

Dividing by $\sqrt{d_h}$ keeps logit variance roughly constant across head widths.

This is a variance-control argument, not a cosmetic constant.

Causality is a constraint on logits



- Position t may use keys $0 \ldots t$.
- Future logits receive $-\infty$ before softmax.
- Masking probabilities after softmax breaks normalization.

Causality test

Change one future token. Earlier output logits must remain identical.

Checkpoint: one query row

Think, then compare

For sequence length four, write the four logits seen by the query at position two. Which entries survive the mask, and across which dimension must softmax normalize?

PART 2

The decoder block

1. Attention from first principles
2. The decoder block
3. Evidence before scale

Multiple heads create multiple retrieval spaces

$$d_h = d/H$$
$$Q, K, V : B \times H \times T \times d_h$$
$$S = QK^\top : B \times H \times T \times T$$
$$\text{concat}(O_h) : B \times T \times d$$

Heads run the same operation with different learned projections. The output projection mixes their results back into the residual stream.

Shape invariant

d must be divisible by H. A transpose mistake can preserve element count while changing semantics.

Useful habit

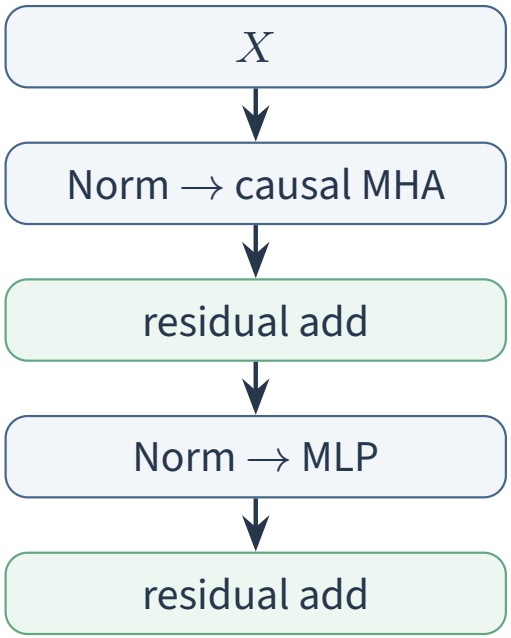
Annotate batch, head, query, key, and channel axes before every reshape.

A pre-norm block has two residual updates

$$X' = X + \mathrm{MHA}(\mathrm{Norm}(X))$$

$$X'' = X' + \mathrm{MLP}(\mathrm{Norm}(X'))$$

- Attention mixes information across positions.
- The MLP transforms each position independently.
- Residual paths preserve a direct information route.



Pre-norm and post-norm are different models

Pre-norm

$$X + F(\mathrm{Norm}(X))$$

The identity path bypasses normalization. Deep optimization is usually more forgiving.

Post-norm

$$\mathrm{Norm}(X + F(X))$$

This is the original Transformer form. It has a different gradient path and training behavior.

Do not describe one convention and implement the other.

The language-model head turns states into a task

$$Z = \mathrm{Norm}(X_N) W_{\text{vocab}}^{\top} \qquad \in \mathbb{R}^{B \times T \times V}$$

$$\mathcal{L} = -\frac{1}{BT} \sum_{b,t} \log p(x_{b,t+1} \mid x_{b,0:t})$$

Each position predicts the next token. The target is the input shifted by one position.

One stream, two views

```
x = tokens[:, :-1]
y = tokens[:, 1:]
```

Common bug

A wrong shift can produce a finite, decreasing loss while training the wrong task.

PART 3

Evidence before scale

1. Attention from first principles
2. The decoder block
3. Evidence before scale

A shape ledger catches semantic bugs

Tensor	Shape	Meaning
token IDs	$B \times T$	discrete input
residual stream	$B \times T \times d$	one state per position
Q, K, V	$B \times H \times T \times d_h$	per-head projections
attention scores	$B \times H \times T \times T$	query-key compatibility
head output	$B \times H \times T \times d_h$	contextualized values
logits	$B \times T \times V$	next-token scores

Review question

Which axis does softmax consume, and which reshape reconstructs d without mixing tokens?

Three tests establish a trustworthy baseline

1. **Shape test**: every intermediate matches the ledger for several B, T, H values.
2. **Causality test**: modifying a future token cannot change earlier logits.
3. **Tiny-batch overfit**: a small model drives loss near zero on one repeated batch.

What each failure means

Shape failure suggests layout. Causality failure suggests masking. Overfit failure suggests optimization, targets, or the end-to-end data path.

The same components support three model families

Encoder-only

Bidirectional attention produces representations for understanding tasks.

Encoder–decoder

A source encoder connects to a causal decoder through cross-attention.

Decoder-only

One causal token stream defines next-token prediction and generation.

Modern LLMs vary attention type, position method, normalization, MLP, and sparsity while retaining this core residual architecture.

Checkpoint: debugging order

Think, then compare

A model runs and loss decreases slowly, but generation copies future tokens from the training sequence. Which test should have caught this, and which tensors would you inspect first?

The durable ideas

1. Attention performs content-addressed mixing through queries, keys, and values.
2. Scaling stabilizes logits. Causal masking protects the prediction task.
3. A decoder block alternates cross-token mixing and per-token transformation.
4. Shapes, causality, and tiny-batch overfitting are the minimum evidence.